



Network Programming (IE2102)
2nd Year, 2st Semester

Assignment

**Design and Implementation of a Secure
Multiprocessor TCP Application Server with
Custom Protocol and Session Management**

Submitted to
Sri Lanka Institute of Information Technology

IT24100936

De Silva K. H. Y. S. T.

In partial fulfillment of the requirements for the
Bachelor of Science Special Honors Degree in Information Technology

2026/04/03

Table of Contents

Declaration.....	4
Introduction.....	5
a) Store the files/ directories	6
b) Host configuration	6
c) Network Configuration	6
d) Server running on correct port	7
e) Build Server	7
f) Run the server	7
g) Run the client.....	7
h) Successful REGISTER and LOGIN	8
i) Failed Login + Lockout proof.....	8
j) Multiple forked processes (ps output).....	9
k) Log file entries	9
l) Proper child cleanup (no Zombie processes).....	10
Conclusion	11
References.....	12

Table of Figures

Figure 1.1: Store the files.....	6
Figure 2.1: Host configuration.....	6
Figure 3.1: Network Configuration.....	6
Figure 4.1: Server running on correct port.....	7
Figure 5.1: Build server	7
Figure 6.1: Run the server.....	7
Figure 7.1: Run the client.....	8
Figure 8.1: Successful REGISTER and LOGIN.....	8
Figure 9.1: Failed Login + Lockout proof	8
Figure 10.1: Multiple forked processes	9
Figure 11.1: Log file entries.....	9
Figure 12.1 Proper child cleanup	10

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text.

Name: De Silva K. H. Y. S. T.

Registration Number: IT24100936

Module: IE2102 - Network Programming

Date: 2026/04/03

Introduction

This report is a design and implementation of a secure multiprocess TCP application server designed to be used in the IE2102 Network Programming assignment. The server was written in C using TCP sockets and process-based concurrency with fork and the client application was written in python. The system enables a custom framing protocol, session token authentication, and abuse protection features, as well as persistent audit logging. The report contains the evidence of implementation and testing screen shots to show that the assignment requirements are followed.

a) Store the files/ directories

This will prepare the needed storage point of the server. The assignment states that the user data should be kept in /srv/ie2102/IT24100936/yehen/. Thus, the required directories were established and access control was set in a way such that the server was in a position to store user credentials, and session-based information in the right place.

```
yehen@yehen:~$ sudo mkdir -p /srv/IE2102/IT24100936
[sudo] password for yehen:
yehen@yehen:~$ sudo chmod -R 777 /srv/IE2102
```

Figure 1.1: Store the files

b) Host configuration

This screenshot shows the hostname of the machine used to run the server and client programs. It helps identify the test environment and confirms the system on which the network programming assignment was implemented and tested.

```
yehen@yehen:~$ hostname
yehen
```

Figure 2.1: Host configuration

c) Network Configuration

This step shows the network details of the Ubuntu system using the ip a command. It confirms the IP configuration of the machine and demonstrates that the environment is properly configured for local TCP communication between the client and server.

```
yehen@yehen:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:4e:3f:13 brd ff:ff:ff:ff:ff:ff
    altnam enp2s1
    inet 192.168.157.128/24 brd 192.168.157.255 scope global dynamic noprefixroute ens33
        valid_lft 1320sec preferred_lft 1320sec
    inet6 fe80::20c:29ff:fe4e:3f13/64 scope link
        valid_lft forever preferred_lft forever
```

Figure 3.1: Network Configuration

d) Server running on correct port

This screen shot indicates that the server is listening on port 50936 which was based on registration number IT24100936 as required by the assignment specification. This demonstrates that the server is bound to the appropriate TCP port and is now ready to receive connections by the clients.

```
yehen@yehen:~$ ss -ltnp | grep 50936
LISTEN 0      20         0.0.0.0:50936  0.0.0.0:*    users:(("server_0936",pid=5032,fd=3))
```

Figure 4.1: Server running on correct port

e) Build Server

This process shows that the compilation of the C server program with the necessary Makefile command was successful. It establishes the fact that the implementation is successfully implemented with the help of gcc, as the assignment presupposes.

```
yehen@yehen:~/IE2102/IT24100936$ make -f Makefile_0936
gcc -Wall -Wextra -pedantic -std=c11 server_0936.c -o server_0936 -lcrypt
```

Figure 5.1: Build server

f) Run the server

In this screenshot, the server binary has been successfully executed. The output shown indicates that the server is started successfully with the desired port number and the appropriate server name SID:1009 that is determined by the middle digits of the registration number.

```
yehen@yehen:~/IE2102/IT24100936$ ./server_0936
Server running on port 50936 with SID:1009
```

Figure 6.1: Run the server

g) Run the client

This is a screenshot of Python client connecting to the TCP server. It also shows the commands supported which can be entered to test registration, login, session handling and other server features.

```
yehen@yehen:~/IE2102/IT24100936$ python3 client_0936.py
Connected to 127.0.0.1:50936
Type commands like:
  REGISTER user123 pass123
  LOGIN user123 pass123
  WHOAMI <token>
  LOGOUT
  PING
Type EXIT to quit.
```

Figure 7.1: Run the client

h) Successful REGISTER and LOGIN

This test proves that it is possible to create a new user account and then authenticate it with the correct username and password. When a successful login is made, the server sends out a session token, and protected commands need the session token. This will ensure that the authentication and session management features are operational.

```
yehen@yehen:~/IE2102/IT24100936$ python3 client_0936.py
Connected to 127.0.0.1:50936
Type commands like:
  REGISTER user123 pass123
  LOGIN user123 pass123
  WHOAMI <token>
  LOGOUT
  PING
Type EXIT to quit.

> REGISTER yehen123 pass123
OK 201 SID:1009 User registered
> LOGIN yehen123 pass123
OK 200 SID:1009 TOKEN yehen123 1775803890 1060891244 656361473
```

Figure 8.1: Successful REGISTER and LOGIN

i) Failed Login + Lockout proof

This test proves the brute-force protection mechanism of the server. In case of repeated wrong credentials, the server blocks the logins and permanently locks the account. This attests to the fact that abuse protection has been set up to minimize repeated malicious logins.

```
> LOGIN nimal123 pass123
ERR 401 SID:1009 Invalid credentials
> LOGIN sunil123 pass123
ERR 401 SID:1009 Invalid credentials
```

Figure 9.1: Failed Login + Lockout proof

j) Multiple forked processes (ps output)

As the screenshot shows, the server can serve several clients at the same time, using the `fork()` system call. A child process is used to handle each session with a client and the parent process is left to accept new connections. This ascertains that the multiprocessing requirement is adhered to.

```
yehen@yehen:~/ie2102/IT24100936$ ps -ef | grep server_0936
root      43120   43021    0 12:17 pts/1    00:00:00 sudo ./server_0936
root      43121   43120    0 12:17 pts/2    00:00:00 sudo ./server_0936
root      43122   43121    0 12:17 pts/2    00:00:00 ./server_0936
root      43204   43122    0 12:28 pts/2    00:00:00 [server_0936] <defunct>
root      43212   43122    0 12:28 pts/2    00:00:00 ./server_0936
yehen     43246   43236    0 12:32 pts/3    00:00:00 grep --color=auto server_0936
```

Figure 10.1: Multiple forked processes

k) Log file entries

This is a screenshot of the audit log file created by the server. The log shows information like time, client IP and port, child process ID, username, issued command and result. This is an affirmation that the continuous logging feature that is demanded by the assignment is put in place.

```
yehen@yehen:~/ie2102/IT24100936$ cat server_IT24100936.log
2026-04-10 12:14:16 | 127.0.0.1:47722 | PID:43067 | USER:- | CMD:REGISTER yehen123 pass123 | RESULT:ERR 500 save-failed
2026-04-10 12:18:00 | 127.0.0.1:41068 | PID:43126 | USER:yehen123 | CMD:REGISTER yehen123 pass123 | RESULT:OK 201 registered
2026-04-10 12:21:30 | 127.0.0.1:41068 | PID:43126 | USER:yehen123 | CMD:LOGIN yehen123 pass123 | RESULT:OK 200 login-success
2026-04-10 12:26:05 | 127.0.0.1:41068 | PID:43126 | USER:nimal123 | CMD:LOGIN nimal123 pass123 | RESULT:ERR 401 invalid-credentials
2026-04-10 12:26:19 | 127.0.0.1:41068 | PID:43126 | USER:sunil123 | CMD:LOGIN sunil123 pass123 | RESULT:ERR 401 invalid-credentials
2026-04-10 12:26:31 | 127.0.0.1:41068 | PID:43126 | USER:- | CMD:LOGIN amal123 pass123 | RESULT:ERR 440 session-expired
2026-04-10 12:27:49 | 127.0.0.1:41068 | PID:43126 | USER:- | CMD:WHOAMI | RESULT:ERR 400 bad-whoami-format
2026-04-10 12:28:36 | 127.0.0.1:52376 | PID:43204 | USER:yehen123 | CMD:LOGIN yehen123 pass123 | RESULT:OK 200 login-success
2026-04-10 12:29:02 | 127.0.0.1:50428 | PID:43212 | USER:yehen123 | CMD:LOGIN yehen123 pass123 | RESULT:OK 200 login-success
2026-04-10 12:29:24 | 127.0.0.1:50428 | PID:43212 | USER:yehen123 | CMD:WHOAMI yehen123_1775804342_1060891244_656361473 | RESULT:OK 200 whoami
```

Figure 11.1: Log file entries

I) Proper child cleanup (no Zombie processes)

This will serve to prove that child processes can be cleaned up properly with SIGCHLD handling and waitpid(). Nonetheless, the existing screenshot must be changed with one demonstrating that there are no zombie entries with the server process, as the current screenshot seems to have a zombie process and fails to serve the need accordingly.

```
yehen@yehen:~/ie2102/IT24100936$ ps -el | grep Z
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
5 Z   0      43204    43122  0  80   0  -   0  -   pts/2        00:00:00 server_0936
```

Figure 12.1 Proper child cleanup

Conclusion

As the conclusion, the system in place illustrates the fundamental principles of TCP socket programming, multiprocessing using fork, protocol framing, authentication, and secure session management. The test results indicate that the server is capable of receiving client connection, handling commands, creating logs, and implementing security measures like validating the login and preventing abuse. More refinement is required to be sure that the end product of child-process cleanup will be a complete illustration of the lack of zombie processes prior to final submission.

References

- **Beej, J.** (2023).
Beej's Guide to Network Programming.
<https://beej.us/guide/bgnet/>
- **Linux Manual Pages**
fork(2), socket(2), recv(2), send(2), waitpid(2)
<https://man7.org/linux/man-pages/>
- **OpenSSL Documentation**
Cryptographic Functions and Secure Hashing
<https://www.openssl.org/docs/>
- **OWASP Foundation** (2023).
Authentication and Session Management Guidelines
<https://owasp.org/>